

Day 12 – Palindrome Number (Part 1)

Aaj ka topic:

Check whether a Number is Palindrome or not

Ye bahut common interview question hai aur logic building ke liye important hai.

Problem Statement

Ek integer n diya hai.

Check karna hai ki number **Palindrome** hai ya nahi.

Palindrome matlab:

Jo number aage se aur piche se same dikhe.

Example 1

Input:
121

Output:
Palindrome

Kyunki

121

↓

121

Same hai.

Example 2

Input:
123

Reverse

321

Different

Output

Not Palindrome

Example 3

Input:
1221

Reverse

1221

Same

Output

Palindrome

Kitni Approaches Hain?

Is problem ke liye actual interviews me mainly **2 approaches** use hoti hain.

Approach 1 — Reverse the Number ☐☐☐☐☐

Ye sabse common aur interview standard approach hai.

Time:

$O(\log N)$

Space:

$O(1)$

Approach 2 — Half Reverse (LeetCode Optimal)

Ye thoda advanced hai.

Isme pura number reverse nahi karte.

Sirf half reverse karte hain.

Ye overflow se bhi bachata hai.

Isko hum **Part 2** me padhenge.

Approach 1 – Reverse the Number

Idea

Reverse Number wala logic use karo.

Fir compare karo.

Agar

`Original == Reverse`

To Palindrome.

Nahi to Not Palindrome.

Example

Input

121

Reverse

121

Compare

`121 == 121`

True

Answer

Palindrome

Input

123

Reverse

321

Compare

```
123 == 321
```

False

Answer

Not Palindrome

Logic

Step 1

Original number save karo.

```
original = n;
```

Step 2

Reverse nikalo.

```
reverse = reverse * 10 + digit;
```

Step 3

Compare karo.

```
if(original == reverse)
```

Palindrome.

Java Code

```
import java.util.Scanner;

public class PalindromeNumber {

    public static boolean isPalindrome(int n) {

        int original = n;
        int reverse = 0;

        while (n != 0) {

            int digit = n % 10;

            reverse = reverse * 10 + digit;

            n = n / 10;
        }

        return original == reverse;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Number : ");

        int n = sc.nextInt();

        if (isPalindrome(n)) {
            System.out.println("Palindrome");
        } else {
            System.out.println("Not Palindrome");
        }
    }
}
```

Line by Line Explanation

Line

```
int original = n;
```

Original number save kar liya.

Example

```
n = 12321
```

↓

```
original = 12321
```

Kyun?

Kyuki loop ke baad

```
n = n / 10;
```

karte karte

```
n
```

↓

```
0
```

ho jayega.

Isliye compare karne ke liye original save karna zaruri hai.

Line

```
int reverse = 0;
```

Reverse number banane ke liye.

Line

```
while(n != 0)
```

Jab tak digits bachi hain.

Line

```
int digit = n % 10;
```

Last digit nikalo.

Line

```
reverse = reverse * 10 + digit;
```

Reverse banao.

Line

```
n = n / 10;
```

Last digit hata do.

Line

```
return original == reverse;
```

Agar dono same hain

```
true
```

warna

```
false
```

Complete Dry Run

Input

1221

Initially

```
original = 1221
```

```
reverse = 0
```

Iteration 1

```
digit = 1
```

```
reverse = 1
```

```
n = 122
```

Iteration 2

```
digit = 2
```

```
reverse = 12
```

```
n = 12
```

Iteration 3

```
digit = 2
```

```
reverse = 122
```

```
n = 1
```

Iteration 4

```
digit = 1
```

```
reverse = 1221
```

```
n = 0
```

Loop end.

Compare

```
1221 == 1221
```

True

Output

Palindrome

Dry Run (Not Palindrome)

Input

123

Reverse

321

Compare

123 == 321

False

Output

Not Palindrome

Time Complexity

Har iteration me ek digit remove hoti hai.

Agar number me d digits hain, to loop d baar chalega.

Aur

$d = \log_{10}(N) + 1$

Isliye

Time Complexity = $O(\log N)$

Space Complexity

Hum sirf 3 variables use kar rahe hain.

- `original`
- `reverse`
- `digit`

Input size badhne par extra memory nahi badhti.

Space Complexity = $O(1)$

Advantages

- Simple aur easy to understand.
 - Interview me sabse common solution.
 - Extra memory nahi lagti.
 - Reverse Number wale concept ka reuse hota hai.
-

Disadvantages

- Agar reverse `int` range se bahar chala gaya to overflow ho sakta hai.
 - Isliye LeetCode me ek aur better approach use hoti hai (Half Reverse), jo hum Part 2 me padhenge.
-

Common Mistakes

Mistake 1

Original save nahi karna.

```
int original = n;
```

Bhool gaye.

Loop ke baad

```
n = 0
```

ho jayega.

Fir compare kis se karoge?

Mistake 2

```
reverse = reverse + digit;
```

☐ Wrong

Correct

```
reverse = reverse * 10 + digit;
```

Mistake 3

Negative numbers ko ignore karna.

Example

-121

Reverse

-121

Mathematically same lag sakta hai.

Lekin interview me generally **negative numbers palindrome nahi maane jaate**, kyunki - sign sirf ek side hota hai.

Edge Cases

Input

0

Output

Palindrome

Input

7

Output

Palindrome

Input

10

Reverse

1

Output

Not Palindrome

Input

1001

Reverse

1001

Output

Palindrome

Interview Me Kaunsi Approach Likhen?

Agar normal palindrome question hai:

☐ **Approach 1 (Reverse the Number)**

Kyun?

- $O(\log N)$ Time
 - $O(1)$ Space
 - Sabse common interview solution
 - Easy to explain
-

Part 2 me hum cover karenge:

- **Approach 2: Half Reverse (Most Optimal / LeetCode Solution)**
- Pura number reverse kyu nahi karte?
- Overflow ka issue kaise avoid hota hai?
- Odd aur Even digits ko kaise handle karte hain?
- Complete dry run aur interview explanation.

Day 12 – Palindrome Number (Part 2)

Approach 2 – Reverse Only Half of the Number ☐ ☐ ☐ ☐ ☐

Ye LeetCode ka official optimal approach hai.

Agar interviewer bole:

"Can you solve it without reversing the entire number?"

to ye approach use karni hai.

Sochne Wali Baat

Approach 1 me hum kya kar rahe the?

12321

↓

Reverse

↓

12321

↓

Compare

Matlab pura number reverse kar rahe the.

Question:

Kya hume pura reverse karna zaruri hai?

Answer:

Nahi.

Sirf **aadha (half)** reverse karke bhi answer mil sakta hai.

Example 1 (Even Digits)

Input

1221

Left Side

12

Right Side

21

Agar right side ko reverse kare

21

↓

12

Dono equal.

Matlab palindrome.

Example 2 (Odd Digits)

12321

Left

12

Middle

3

Right

21

Right reverse

21

↓

12

Middle digit ko ignore kar sakte hain.

Again palindrome.

Main Idea

Hum last digits nikalte jayenge.

Lekin pura reverse nahi banayenge.

Sirf tab tak reverse banayenge jab tak:

```
reverse < n
```

Ye condition bahut important hai.

Example

Input

12321

Initially

```
n = 12321
```

```
reverse = 0
```

Iteration 1

Digit

1

Reverse

1

Remaining Number

1232

Iteration 2

Digit

2

Reverse

12

Remaining Number

123

Iteration 3

Digit

3

Reverse

123

Remaining Number

12

Ab dekho

```
reverse = 123
```

```
n = 12
```

Ab

```
reverse > n
```

Matlab half complete.

Loop stop.

Odd Digits Problem

Ab compare karte hain.

```
reverse = 123
```

```
n = 12
```

Equal nahi.

Kyun?

Middle digit

3

extra aa gaya.

Isliye

```
reverse / 10
```

kar dete hain.

123

↓

12

Ab compare

```
12 == 12
```

True.

Even Digits

Input

1221

Iterations

reverse = 1

n =122

↓

reverse =12

n =12

Stop.

Compare

12 ==12

True.

Is case me /10 ki zarurat nahi.

Java Code

```
import java.util.Scanner;

public class PalindromeHalfReverse {

    public static boolean isPalindrome(int n) {

        // Negative numbers palindrome nahi hote
        if (n < 0) {
            return false;
        }

        // Agar last digit 0 hai aur number 0 nahi hai
        if (n != 0 && n % 10 == 0) {
            return false;
        }

        int reverse = 0;

        while (n > reverse) {

            int digit = n % 10;

            reverse = reverse * 10 + digit;

            n = n / 10;
        }

        return (n == reverse) || (n == reverse / 10);
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Number : ");

        int n = sc.nextInt();

        if (isPalindrome(n)) {
            System.out.println("Palindrome");
        } else {
            System.out.println("Not Palindrome");
        }
    }
}
```

Line by Line Explanation

Line

```
if(n < 0)
```

Example

-121

Palindrome nahi.

Immediately

```
return false;
```

Ye Condition Bahut Important Hai

```
if(n != 0 && n % 10 == 0)
```

Example

10

Reverse

01

Number

1

Ho gaya.

Equal nahi.

Isliye

```
return false;
```

Exception

0

Palindrome hai.

Isliye

```
n != 0
```

likha hai.

Reverse Variable

```
int reverse = 0;
```

Sirf half reverse karega.

Loop

```
while(n > reverse)
```

Ye pura approach ka heart hai.

Jab tak left side badi hai

tab tak reverse banao.

Jaise hi reverse bada ya equal ho

Stop.

Last Digit

```
digit = n % 10;
```

Same Reverse Number logic.

Reverse

```
reverse = reverse * 10 + digit;
```

Half reverse banta jayega.

Remove Last Digit

```
n = n / 10;
```

Left side chhoti hoti jayegi.

Final Comparison

```
return (n == reverse)
        ||
        (n == reverse / 10);
```

Yahi sabse important line hai.

Ye do Conditions Kyu?

Case 1

Even Digits

1221

End me

n = 12

reverse = 12

Compare

12 == 12

True.

Case 2

Odd Digits

```
12321
```

End

```
n =12
```

```
reverse =123
```

Middle digit

```
3
```

Ignore

```
123/10
```

```
↓
```

```
12
```

Compare

```
12==12
```

True.

Complete Dry Run (Even)

Input

```
1221
```

Initially

```
n=1221
```

```
reverse=0
```

Iteration 1

```
digit=1
```

```
reverse=1
```

```
n=122
```

Iteration 2

digit=2

reverse=12

n=12

Loop Stop

Compare

12==12

Answer

Palindrome

Complete Dry Run (Odd)

Input

12321

Initially

n=12321

reverse=0

Iteration 1

digit=1

reverse=1

n=1232

Iteration 2

digit=2

reverse=12

n=123

Iteration 3

digit=3

reverse=123

`n=12`

Stop.

Compare

`12==123`

False.

Second Compare

`12==123/10`

↓

`12==12`

True.

Answer

Palindrome

Time Complexity

Har iteration me ek digit remove hoti hai.

Lekin yahan hum **sirf half digits** process karte hain.

Agar total digits `d` hain:

`Iterations ≈ d / 2`

Mathematically:

`O(log N / 2)`

Big-O notation me constant ($1/2$) ignore hota hai.

Isliye final:

`Time Complexity = O(log N)`

Space Complexity

Sirf 2–3 variables use ho rahe hain.

Koi extra array ya string nahi.

Space Complexity = $O(1)$

Advantages

- ☐ Overflow ka risk nahi.
 - ☐ Sirf half digits process hoti hain.
 - ☐ LeetCode official optimal solution.
 - ☐ $O(1)$ extra space.
 - ☐ Advanced interview me impress karta hai.
-

Disadvantages

- Logic beginners ke liye thoda tricky hai.
 - `reverse / 10` wali condition pehle samajhna mushkil lag sakta hai.
-

Common Mistakes

Mistake 1

```
while(n != 0)
```

☐ Wrong

Is approach me

```
while(n > reverse)
```

likhna hai.

Mistake 2

Odd digits ke liye

```
return n==reverse;
```

Sirf ye likhna.

☐ Wrong

`reverse / 10` bhi check karna padega.

Mistake 3

Last digit 0 wale numbers ko handle na karna.

Example:

```
10  
100  
120
```

Ye palindrome nahi hote.

Interview Me Kaunsi Approach Likhen?

☐ **Approach 1 (Reverse Number)**

- Simple
 - Easy explain karna
 - Most expected
-

Agar interviewer bole:

- "Can you optimize?"
- "Avoid overflow."
- "Don't reverse the entire number."

☐ **Approach 2 (Half Reverse)**

Ye advanced solution hai aur strong understanding dikhata hai.